

Arquitectura Sistémica y Fundamentos de la Metodología DevObs: Guía Maestra para la Construcción de un Entorno de Ingeniería Basado en la Observabilidad y la Documentación como Código

El paradigma contemporáneo en el desarrollo de software ha trascendido la mera integración de equipos de desarrollo y operaciones para dar paso a un modelo de gestión mucho más profundo, proactivo y fundamentado en la evidencia de los datos: la metodología DevObs. Esta evolución, que amalgama las prácticas consolidadas de DevOps con la disciplina emergente de la observabilidad (Observability), no solo representa un cambio en el conjunto de herramientas tecnológicas, sino una transformación radical en la cultura organizacional y en la forma en que el conocimiento se captura, se procesa y se utiliza para la toma de decisiones estratégicas. En un proyecto con una ventana temporal de cuatro semanas, la cimentación de este entorno durante la fase inicial resulta determinante para el éxito del producto final, permitiendo que un equipo de ocho personas opere de manera coordinada, transparente y eficiente.

La metodología DevObs se distancia del monitoreo tradicional, que históricamente se ha centrado en el estado binario de los sistemas (activo o inactivo), para enfocarse en la comprensión profunda de los estados internos de una aplicación mediante el análisis de sus salidas externas.¹ Para los gestores de documentación, este cambio implica que su labor ya no consiste exclusivamente en la redacción de manuales estáticos, sino en el diseño y mantenimiento de una infraestructura de información viva que se alimenta de la telemetría del sistema y que se integra de manera orgánica en el ciclo de vida del desarrollo de software.³

Evolución del Paradigma: De DevOps a la Hegemonía de la Observabilidad

Para comprender la estructura de DevObs, es imperativo analizar su origen como una evolución de la metodología ágil y de DevOps. Mientras que el enfoque ágil rompió con la rigidez de la metodología en cascada (Waterfall) para introducir iteraciones rápidas y feedback constante, DevOps eliminó los silos entre los equipos de desarrollo (Dev) y operaciones (Ops), fomentando una cultura de responsabilidad compartida y automatización a través de pipelines de integración y entrega continua (CI/CD).⁵ DevObs representa el siguiente nivel de madurez, donde la capacidad de observar el comportamiento del sistema en producción se convierte en un requisito de diseño fundamental, permitiendo a los ingenieros identificar no solo qué ha

fallado, sino por qué ha fallado, incluso en situaciones de fallos desconocidos o imprevistos.²

La observabilidad en DevObs no debe entenderse como un añadido posterior al desarrollo, sino como una propiedad intrínseca del sistema que se construye desde la primera semana de trabajo. Esta propiedad permite que los desarrolladores dejen de lanzar código "sobre la valla" hacia el equipo de operaciones, asumiendo en su lugar la responsabilidad total del ciclo de vida del software.⁹ En este contexto, la infraestructura funcional que se debe levantar no solo incluye servidores y bases de datos, sino también los mecanismos de recolección de señales telemétricas y los repositorios de conocimiento que permitirán al equipo navegar la complejidad técnica del proyecto.

Comparación Crítica de los Modelos de Gestión de Software

Dimensión de Análisis	Metodología Waterfall	Metodología DevOps	Metodología DevObs
Enfoque Principal	Secuencial y por hitos rígidos.	Automatización y colaboración Dev+Ops.	Observabilidad proactiva y toma de decisiones por datos.
Rol de la Documentación	Artefacto final, extenso y estático.	Documentación técnica mínima y manual.	Documentación como Código (DaC) y telemetría integrada.
Gestión de Errores	Reactiva y basada en manuales.	Fallos rápidos en el pipeline CI/CD.	Diagnóstico de "incógnitas desconocidas" en tiempo real.
Estructura del Equipo	Silos funcionales aislados.	Equipos integrados y multifuncionales.	Equipos orientados a la fiabilidad y transparencia absoluta.
Velocidad de Entrega	Lenta, con ciclos de meses o años.	Rápida, con despliegues frecuentes.	Continua, estable y altamente predecible.

La adopción de DevObs por parte de los dos gestores de documentación requiere entender

que ellos son los arquitectos de un ecosistema de información donde los requerimientos funcionales y no funcionales, ya planteados en la semana uno, se convierten en las bases para definir qué métricas deben ser observadas y qué procesos deben ser documentados con mayor rigurosidad.⁵

Pilares Arquitectónicos de la Observabilidad en DevOps

La infraestructura funcional de un entorno DevOps se sostiene sobre la capacidad de capturar tres tipos fundamentales de datos telemétricos: métricas, registros (logs) y trazas (tracing). Estos pilares proporcionan la visibilidad necesaria para que los cuatro programadores y el gestor de proyecto comprendan el rendimiento, la seguridad y la estabilidad del sistema sin necesidad de recurrir a suposiciones.¹

El flujo de las métricas cuantitativas

Las métricas son representaciones numéricas de datos recolectados en intervalos regulares que permiten medir el rendimiento del host, la aplicación y la red.¹⁴ En un entorno DevOps, las métricas son esenciales para establecer umbrales de alerta y monitorear la salud general de la infraestructura. Para los documentadores, es vital categorizar estas métricas de modo que el equipo sepa qué señales observar para cada componente del sistema.¹⁴

- **Métricas del Host:** Incluyen el uso de CPU, memoria ram, almacenamiento en disco y rendimiento de la red. Son fundamentales para la gestión de la capacidad y el escalado de la infraestructura.¹⁴
- **Métricas de Aplicación:** Se centran en los tiempos de respuesta, las tasas de error por cada mil solicitudes y la latencia de las transacciones.¹⁵
- **Métricas de Dependencias Externas:** Monitorean la disponibilidad y los tiempos de respuesta de APIs de terceros o servicios en la nube de los que depende el sistema.¹⁴

La narrativa de los registros (Logs)

A diferencia de las métricas, que indican qué está ocurriendo de forma cuantitativa, los registros proporcionan el contexto cualitativo para entender por qué ha ocurrido un evento específico.¹⁴ Un registro efectivo en un entorno DevOps debe estar estructurado y contener marcas de tiempo precisas, identificadores de transacción y niveles de severidad claramente definidos.¹⁴ La infraestructura de logs debe ser centralizada, permitiendo que los gestores de documentación diseñen guías de solución de problemas basadas en los mensajes de error recurrentes que el sistema emite.¹³

El rastreo de la causalidad mediante las Trazas (Tracing)

En arquitecturas de microservicios o sistemas distribuidos, una sola solicitud de usuario puede

atravesar múltiples componentes. Las trazas permiten seguir el recorrido de esa solicitud, identificando exactamente dónde se producen los cuellos de botella o las fallas de comunicación entre servicios.⁶ La implementación de trazas distribuidas requiere una instrumentación cuidadosa del código por parte de los programadores, pero una vez establecida, ofrece una visibilidad sin precedentes sobre la causa raíz de los errores complejos.¹⁶

Tipo de Telemetría	Pregunta que Resuelve	Características del Dato	Herramientas Ejemplo
Métricas	¿Qué está pasando y cuánto?	Datos numéricos agregados, bajo costo de almacenamiento.	Prometheus, Grafana, Datadog. ²¹
Logs	¿Por qué ocurrió esto específicamente?	Texto estructurado/no estructurado, alta verbosidad.	Stack ELK (Elasticsearch, Logstash, Kibana). ¹⁹
Trazas	¿Dónde se rompió la cadena de llamadas?	Identificadores de solicitud correlacionados a través de servicios.	Jaeger, Zipkin, Dynatrace. ¹⁶

La Metodología Documentation as Code (DaC) como Eje Central

Para que los dos gestores de documentación puedan guiar al equipo de ocho personas, deben implementar el concepto de Documentation as Code (DaC). Esta filosofía propone que la documentación se trate con la misma rigurosidad que el código fuente de la aplicación, utilizando las mismas herramientas y procesos que los programadores.³ Al adoptar DaC, la documentación deja de estar atrapada en sistemas de gestión de contenido (CMS) aislados o documentos de Word perdidos en carpetas compartidas, para integrarse directamente en el repositorio de Git del proyecto.³

Ventajas Estratégicas del Enfoque DaC

La implementación de DaC durante la semana uno y dos establece una base sólida por varias razones fundamentales. En primer lugar, fomenta la colaboración al permitir que tanto

desarrolladores como técnicos utilicen el mismo flujo de trabajo: crear una rama (branch), realizar cambios, proponer un Pull Request y pasar por una revisión por pares.³ En segundo lugar, garantiza la sincronización entre el código y su descripción técnica; si una nueva funcionalidad es integrada al sistema, su documentación correspondiente debe formar parte del mismo envío de código para que el pipeline de CI/CD lo considere un éxito.²⁶

Componentes Técnicos de la Infraestructura de Documentación

Para levantar esta infraestructura funcional, los gestores de documentación deben configurar los siguientes elementos:

1. **Sistemas de Control de Versiones (VCS):** Generalmente Git, alojado en plataformas como GitHub, GitLab o Azure DevOps. Este es el corazón de la colaboración y el historial de cambios.³
2. **Lenguajes de Marcado Ligero:** Se recomienda el uso de Markdown o AsciiDoc debido a su simplicidad y capacidad de ser leídos tanto por humanos como por herramientas de automatización.²⁴
3. **Generadores de Sitios Estáticos (SSG):** Herramientas como MkDocs, Hugo o Docusaurus que transforman los archivos de marcado en un portal de documentación web profesional, buscable y fácil de navegar.²⁵
4. **Validadores Automáticos (Linters):** Scripts que verifican automáticamente errores ortográficos, enlaces rotos o cumplimiento de guías de estilo antes de que la documentación sea publicada.²⁵

Al integrar estos componentes, los documentadores pueden asegurar que la información técnica sea una "fuente única de verdad" (Single Source of Truth), eliminando la duplicidad de información y asegurando que cualquier miembro del equipo pueda acceder a la guía más actualizada de manera instantánea.³

Dinámica y Responsabilidades del Equipo de Ocho Personas en DevOps

La estructura del equipo (1 Gestor de Proyecto, 1 Asistente, 2 Documentadores, 4 Programadores) es ideal para implementar una cultura DevOps de alta intensidad si se definen los roles de manera sistémica. La clave no es asignar tareas aisladas, sino fomentar la multifuncionalidad y la eliminación de barreras de comunicación.⁵

Los Gestores de Documentación como Facilitadores de Conocimiento

En este modelo, los documentadores no solo escriben; son responsables de la gobernanza de la información y de la infraestructura de conocimiento. Su labor incluye definir las plantillas de documentación, supervisar que los programadores documenten sus APIs y procesos de manera correcta, y gestionar el pipeline que despliega la documentación.¹³ Actúan como un

puente entre la complejidad técnica de los programadores y la necesidad de claridad del gestor de proyecto y los stakeholders externos.¹³

Los Programadores y la Responsabilidad sobre la Telemetría

Los cuatro programadores en un entorno DevObs deben adoptar la mentalidad de "si lo construyes, lo ejecutas y lo observas". Esto significa que su trabajo no termina cuando el código es funcional, sino cuando está debidamente instrumentado para emitir métricas, logs y trazas que permitan su monitoreo en producción.¹⁰ También deben participar activamente en la documentación técnica, escribiendo los borradores iniciales de las especificaciones que luego los gestores de documentación refinarán para asegurar consistencia y calidad editorial.⁴

El Gestor de Proyecto y el Asistente como Orquestadores

El gestor de proyecto utiliza los tableros de observabilidad y la documentación de progreso para supervisar el avance real del proyecto frente a los requerimientos funcionales y no funcionales.⁵ El asistente, por su parte, juega un rol crucial en la logística de la información: asegurando que las herramientas de colaboración (como Jira, Slack o repositorios de documentos) estén organizadas, gestionando los permisos de acceso y apoyando en la generación de reportes de cumplimiento basados en los datos del sistema.¹⁰

Rol en el Equipo	Responsabilidad Principal en DevObs	Interacción Clave
Gestor de Proyecto	Supervisión estratégica y gestión de riesgos basada en datos. ¹⁰	Alínea objetivos de negocio con métricas de sistema.
2 Documentadores	Arquitectura de la infraestructura DaC y gobernanza de la información. ²⁵	Revisan el código para asegurar que la documentación esté al día.
4 Programadores	Desarrollo de funcionalidades e instrumentación de observabilidad. ¹⁰	Proveen datos técnicos precisos para la base de conocimientos.
1 Asistente	Soporte administrativo, gestión de accesos y logística de herramientas. ⁴²	Facilita el flujo de trabajo sin fricciones operativas.

Infraestructura de CI/CD: El Sistema Nervioso de DevOps

Para que un entorno DevOps sea funcional, es necesario levantar una tubería de integración y entrega continua que automatice el ciclo de vida tanto del software como de la documentación. El pipeline de CI/CD es el mecanismo que garantiza que cada pequeño cambio sea probado, validado y desplegado de forma segura y predecible.⁵

Fases Críticas del Pipeline Integrado

1. **Validación de Código y Docs:** Cada vez que un programador o documentador realiza un "commit", el pipeline debe ejecutar pruebas automáticas. Para el código, esto incluye pruebas unitarias y de integración; para la documentación, incluye validaciones de formato y pruebas de estilo.⁵
2. **Generación de Artefactos:** El sistema construye la aplicación y, simultáneamente, genera el sitio estático de la documentación a partir de los archivos Markdown o AsciiDoc.³²
3. **Despliegue en Entornos de Staging:** Antes de llegar a producción, tanto el software como su documentación se despliegan en un entorno espejo para una validación final. Aquí es donde la observabilidad permite verificar que el nuevo código no degrada el rendimiento del sistema.⁴³
4. **Promoción a Producción:** Una vez aprobada, la actualización se despliega automáticamente (CD). La documentación actualizada se publica en el portal, asegurando que los usuarios finales o el equipo de soporte tengan acceso inmediato a la información sobre las nuevas características o cambios.²⁶

Este flujo de trabajo elimina la necesidad de intervenciones manuales propensas al error humano y reduce drásticamente el tiempo de entrega al mercado (Time-to-Market), permitiendo que un proyecto de cuatro semanas mantenga un ritmo constante de progreso.⁷

Métricas de Servicio: SLIs, SLOs y el Control de Calidad

Una infraestructura DevOps no solo debe funcionar, sino que debe cumplir con niveles específicos de calidad acordados. Para ello, se deben definir indicadores de nivel de servicio (SLIs) y objetivos de nivel de servicio (SLOs).³⁸

Definición de Indicadores de Nivel de Servicio (SLI)

Los SLIs son las métricas críticas que definen la salud del sistema desde la perspectiva del usuario final. Ejemplos comunes incluyen la latencia de las solicitudes, la tasa de disponibilidad del sistema y el rendimiento de las transacciones.¹⁸ Para los documentadores, documentar estos SLIs es fundamental para que todo el equipo hable el mismo lenguaje respecto a lo que

significa que el sistema esté "funcionando bien".³⁸

Establecimiento de Objetivos de Nivel de Servicio (SLO)

Los SLOs son los valores o rangos de valores que el equipo se compromete a mantener para cada SLI. Por ejemplo, un SLO común podría ser que "el 99.9% de las solicitudes a la API deben responder en menos de 200 milisegundos durante un periodo de 30 días".¹⁸ La importancia de los SLOs en DevObs radica en que proporcionan una guía objetiva para decidir cuándo el equipo puede enfocarse en nuevas funcionalidades y cuándo debe detenerse para mejorar la estabilidad del sistema.³⁸

El Concepto de Presupuesto de Error (Error Budget)

El presupuesto de error es el margen permitido de fallos antes de que el SLO sea violado. Si un sistema tiene un SLO de disponibilidad del 99.9%, el presupuesto de error es del 0.1%, lo que equivale a aproximadamente 43 minutos de inactividad permitida por mes.⁴⁹ En DevObs, el presupuesto de error fomenta una toma de riesgos calculada y proporciona una métrica clara para la priorización del trabajo de los programadores y el gestor de proyecto.³⁸

Métrica	Definición Conceptual	Ejemplo Práctico
SLI	Medición cuantitativa del servicio. ⁴⁸	Latencia de carga de la página principal.
SLO	Meta de rendimiento para el SLI. ⁴⁸	95% de las páginas cargan en < 2 segundos.
SLA	Acuerdo contractual con consecuencias legales. ⁴⁸	Disponibilidad del 99.9% o compensación financiera.
Error Budget	Margen de error aceptable antes de actuar. ⁴⁹	5% de las solicitudes pueden fallar sin violar el SLO.

Gestión de la Configuración y Seguridad en el Entorno DevObs

Un entorno funcional debe ser seguro por diseño. La integración de la seguridad en el flujo de trabajo de DevOps se conoce comúnmente como DevSecOps, un subcomponente vital de DevObs.¹² La infraestructura debe incluir herramientas de escaneo automático de

vulnerabilidades en el código, gestión segura de secretos (como claves de API y contraseñas de bases de datos) y auditorías continuas de cumplimiento normativo.¹²

Infraestructura como Código (IaC)

Para que el entorno sea reproducible y escalable, la infraestructura física o en la nube debe ser definida mediante código utilizando herramientas como Terraform, Ansible o AWS CloudFormation.²¹ Esto permite que los gestores de documentación puedan ver exactamente cómo está configurado el entorno simplemente leyendo el código de infraestructura, lo que facilita la creación de diagramas de arquitectura y guías de despliegue precisas.¹²

Seguridad Desplazada a la Izquierda (Shift-Left Security)

DevOps promueve que las pruebas de seguridad se realicen lo más temprano posible en el ciclo de desarrollo. Los programadores deben recibir feedback inmediato sobre posibles fallos de seguridad en su código directamente en su entorno de desarrollo o a través del pipeline de CI/CD.¹² Al documentar estos requisitos de seguridad desde la semana uno, los documentadores aseguran que el equipo no deje la protección del sistema como un pensamiento de último minuto.¹³

Estructuración del Ecosistema de Información para los Documentadores

Para que los dos gestores de documentación puedan guiar efectivamente al equipo de ocho personas, deben organizar la información en capas lógicas que respondan a diferentes necesidades. Un enfoque recomendado es el marco de trabajo Diátaxis, que divide la documentación en cuatro orientaciones: tutoriales, guías prácticas, referencias técnicas y explicaciones conceptuales.³³

Orientación por Necesidad del Usuario

1. **Tutoriales:** Enfocados en el aprendizaje inicial de los miembros del equipo que se incorporan al proyecto. Deben guiar al usuario paso a paso en una tarea específica, como configurar su entorno local de desarrollo.³³
2. **Guías Prácticas (How-to Guides):** Instrucciones directas para resolver problemas específicos o realizar tareas recurrentes, como "cómo desplegar una actualización de emergencia" o "cómo crear un nuevo dashboard de métricas".³³
3. **Referencias Técnicas:** Documentación exhaustiva y a menudo generada automáticamente sobre las APIs, los esquemas de bases de datos y los parámetros de configuración. Es la capa de mayor detalle técnico.³³
4. **Explicaciones Conceptuales:** Documentos que proporcionan el contexto y el "porqué" de las decisiones arquitectónicas. Ayudan a los miembros del equipo a comprender la lógica detrás del sistema, como la elección de un modelo de base de datos específico o la

estrategia de microservicios adoptada.³³

Gestión de Versiones de Documentación

En un entorno DevOps donde el software cambia constantemente, es imperativo que la documentación también esté versionada. El uso de ramas en Git permite que existan diferentes versiones de la documentación simultáneamente, correspondiendo a las versiones de producción, pruebas y desarrollo del software.²⁸ Esto evita confusiones peligrosas, como que un desarrollador consulte la documentación de una versión futura mientras intenta depurar un problema en la versión actual de producción.²⁸

Cimentación de la Infraestructura en las Semanas 1 y 2

Al concluir la semana uno, el equipo ya posee requerimientos funcionales y un avance semanal. El siguiente paso crítico para los documentadores es habilitar el entorno técnico que permitirá que la documentación sea "agradable" y eficiente. Este proceso no se trata de escribir contenido, sino de "armar el andamiaje".²⁷

Roadmap para los Gestores de Documentación

- **Día 1-2: Selección y Configuración del Toolstack.** Elegir el generador de sitios estáticos (ej. MkDocs con el tema Material) y configurar el repositorio de Git con la estructura de carpetas necesaria (/src para el código y /docs para la documentación).²⁵
- **Día 3: Implementación del Pipeline de CI/CD para Docs.** Configurar una acción automática (como GitHub Actions) que valide y publique el sitio de documentación cada vez que se realice un cambio en la rama principal. Esto garantiza visibilidad inmediata del avance.²⁹
- **Día 4: Definición de Estándares de Telemetría.** Colaborar con los programadores para definir el formato de los logs y los nombres de las métricas clave. Esto asegura que la observabilidad sea coherente a través de todos los servicios.¹⁴
- **Día 5: Creación de Plantillas y Guías de Estilo.** Establecer las reglas de juego para que los programadores sepan cómo documentar sus contribuciones. Una plantilla simple para mensajes de commit y Pull Requests mejora drásticamente la trazabilidad.²⁸

Al final de este proceso, el equipo de ocho personas tendrá un entorno funcional donde la información fluye de manera natural. El gestor de proyecto podrá ver el avance real en el portal de docs, los programadores tendrán un lugar centralizado para sus referencias técnicas, y los documentadores podrán centrarse en elevar la calidad del conocimiento capturado en lugar de luchar con herramientas manuales e ineficientes.³

El Rol de la Automatización y la Inteligencia Artificial en DevOps

Hacia el futuro del proyecto, la infraestructura puede potenciarse integrando herramientas de inteligencia artificial que asistan tanto en la programación como en la documentación. Herramientas como Copilot para código o asistentes de IA integrados en el sistema de gestión de documentos (DMS) pueden automatizar la generación de resúmenes, la detección de redundancias y la traducción técnica.¹¹ Sin embargo, la base sólida reside siempre en la estructura humana y los procesos técnicos definidos en las fases iniciales.³

La metodología DevObs, al unir la agilidad de DevOps con la profundidad analítica de la observabilidad y el rigor de la documentación como código, crea un ecosistema donde el error no es una catástrofe, sino una oportunidad de aprendizaje documentada y medible. Para el equipo de ocho personas, esta es la ruta hacia la entrega de un sistema de alta calidad en un plazo de cuatro semanas, asegurando que el conocimiento generado durante el proceso perdure mucho después de que el código haya sido desplegado.²

Síntesis de la Infraestructura de Soporte para la Semana 2-4

Componente	Estado Requerido para Inicio de Documentación	Responsable de Mantenimiento
Repositorio VCS	Estructurado con protección de ramas y linters de markdown. ²⁸	Gestores de Documentación.
Portal de Documentación	Desplegado automáticamente y accesible por todo el equipo. ²⁷	Gestores de Documentación.
Stack de Observabilidad	Dashboards básicos con las "Cuatro Señales de Oro" (Latencia, Tráfico, Errores, Saturación). ²	Programadores / Documentadores.
Pipeline CI/CD	Flujo funcional que bloquea fusiones de código sin documentación técnica adjunta. ²⁶	Programadores / Gestor de Proyecto.
Guía de Contribución	Documento claro sobre	Gestores de

	cómo usar Git, Markdown y el flujo de revisión por pares. ²⁸	Documentación / Asistente.
--	---	----------------------------

Con estos cimientos establecidos, los dos gestores de documentación se transforman en líderes técnicos de la información, guiando al equipo no a través de imposiciones, sino a través de un sistema diseñado para facilitar el éxito colectivo. La metodología DevObs se convierte así en la brújula que permite al equipo de ocho personas navegar con confianza hacia la culminación del proyecto, transformando los requerimientos funcionales y no funcionales en una realidad técnica robusta, observable y perfectamente documentada.²

Obras citadas

1. Comprender las métricas de observabilidad: tipos, señales de oro y mejores prácticas, fecha de acceso: febrero 27, 2026, <https://www.elastic.co/es/blog/observability-metrics>
2. Observability vs. monitoring in DevOps - GitLab, fecha de acceso: febrero 27, 2026, <https://about.gitlab.com/blog/observability-vs-monitoring-in-devops/>
3. What is Docs as Code? Your Guide to Modern Technical Documentation - Kong Inc., fecha de acceso: febrero 27, 2026, <https://konghq.com/blog/learning-center/what-is-docs-as-code>
4. Beginner's Guide to Documentation as Code - DEV Community, fecha de acceso: febrero 27, 2026, <https://dev.to/giladmaayan/beginners-guide-to-documentation-as-code-11o1>
5. ¿Qué es DevOps? - IBM, fecha de acceso: febrero 27, 2026, <https://www.ibm.com/mx-es/think/topics/devops>
6. ¿En qué consisten las DevOps? - AWS, fecha de acceso: febrero 27, 2026, <https://aws.amazon.com/es/devops/what-is-devops/>
7. DevOps: qué es, cómo funciona y ventajas - BBVA NOTICIAS, fecha de acceso: febrero 27, 2026, <https://www.bbva.com/es/innovacion/devops-que-es-como-functiona-y-ventajas/>
8. Why observability and developer self-service are key trends in DevOps, fecha de acceso: febrero 27, 2026, <https://platformengineering.org/talks-library/observability-and-developer-self-service>
9. Phrases in computing that might need retiring - Hacker News, fecha de acceso: febrero 27, 2026, <https://news.ycombinator.com/item?id=32273933>
10. International DevOps Certification Academy™ What Are The Roles In Your DevOps Organization?, fecha de acceso: febrero 27, 2026, https://www.devops-certification.org/What_Are_The_Roles_In_Your_DevOps_Organization.php
11. DevOps: The Complete Guide To Culture, Technology, And Tools | - Octopus Deploy, fecha de acceso: febrero 27, 2026, <https://octopus.com/devops/devops-approach/>
12. ¿Qué es DevOps? Principios, ventajas y herramientas - SentinelOne, fecha de

acceso: febrero 27, 2026,

<https://www.sentinelone.com/es/cybersecurity-101/cybersecurity/what-is-devops/>

13. ¿Qué hace un DevOps? ¡Descubre todas sus funciones! - Tokio School, fecha de acceso: febrero 27, 2026,
<https://www.tokioschool.com/noticias/devops-funciones/>
14. Tres pilares de la observabilidad: registros, métricas y rastreos | IBM, fecha de acceso: febrero 27, 2026,
<https://www.ibm.com/es-es/think/insights/observability-pillars>
15. Los 3 pilares de la observabilidad: logs, métricas y rastreos unificados | Elastic Blog, fecha de acceso: febrero 27, 2026,
<https://www.elastic.co/es/blog/3-pillars-of-observability>
16. Tres pilares de la observabilidad: registros, métricas y rastreos | IBM, fecha de acceso: febrero 27, 2026,
<https://www.ibm.com/mx-es/think/insights/observability-pillars>
17. Observability-Driven Development Explained: 8 Steps for ODD Success | Splunk, fecha de acceso: febrero 27, 2026,
https://www.splunk.com/en_us/blog/learn/odd-observability-driven-development.html
18. SLOs and SLIs for Databases: The Definitive Guide to Unbeatable Cloud Performance and Peace for Your DevOps! - dbsnOOop, fecha de acceso: febrero 27, 2026, <https://dbsnoop.com/slos-and-slis-for-databases-performance/>
19. Los pilares de la observabilidad: logs, métricas y trazas - ToBeIT, fecha de acceso: febrero 27, 2026,
<https://tobeit.es/los-pilares-de-la-observabilidad-logs-metricas-y-trazas/>
20. Resilient software delivery through observability-driven development | EY - India, fecha de acceso: febrero 27, 2026,
https://www.ey.com/en_in/insights/cybersecurity/resilient-software-delivery-through-observability-driven-development
21. DevOps | PDF | Version Control | System Software - Scribd, fecha de acceso: febrero 27, 2026, <https://www.scribd.com/document/990256113/DevOps>
22. Welcome to DevOps in Softzino Technologies, fecha de acceso: febrero 27, 2026,
<https://softzino.com/services/devops-services>
23. Live Debugger | Dynatrace Hub, fecha de acceso: febrero 27, 2026,
<https://www.dynatrace.com/hub/detail/live-debugger/>
24. Docs as Code - Write the Docs, fecha de acceso: febrero 27, 2026,
<https://www.writethedocs.org/guide/docs-as-code.html>
25. Docs as Code as a new industry standard in documentation - ELEKS, fecha de acceso: febrero 27, 2026, <https://eleks.com/blog/docs-as-code/>
26. Docs-as-code: What is it, how it works, and ways to start | Fern, fecha de acceso: febrero 27, 2026, <https://buildwithfern.com/post/docs-as-code>
27. Documentation as Code: How Agile Development Implements the One Source of Truth Philosophy - SCAND, fecha de acceso: febrero 27, 2026,
<https://scand.com/company/blog/documentation-as-code-agile-one-source-of-truth/>

28. What Is Documentation as Code and Why Do You Need It? - ClickHelp, fecha de acceso: febrero 27, 2026,
<https://clickhelp.com/clickhelp-technical-writing-blog/what-is-documentation-as-code-and-why-do-you-need-it/>
29. Docs-as-Code and DevOps Success. Why is Documentation so Bad? | by Michael Ryan, fecha de acceso: febrero 27, 2026,
<https://medium.com/@michaeljamesryan/docs-as-code-and-devops-success-871bb174631d>
30. Devops | PDF | Version Control | Software Engineering - Scribd, fecha de acceso: febrero 27, 2026, <https://www.scribd.com/document/444456778/Devops>
31. The Complete Version Control Guide for Technical Writers in Docs-as-Code | by Balu Kosuri, fecha de acceso: febrero 27, 2026,
<https://medium.com/@k.balu124/the-complete-version-control-guide-for-technical-writers-in-docs-as-code-a79f1d1aa556>
32. Docs as Code - The New Way of Creating Documentation - adoc Studio, fecha de acceso: febrero 27, 2026, <https://www.adoc-studio.app/blog/docs-as-code>
33. Build Your Python Project Documentation With MkDocs - Real Python, fecha de acceso: febrero 27, 2026,
<https://realpython.com/python-project-documentation-with-mkdocs/>
34. Docs as Code: The Future of Seamless Documentation - TimelyText, fecha de acceso: febrero 27, 2026,
<https://www.timelytext.com/docs-as-code-the-future-of-seamless-documentation/>
35. Impulsa la agilidad y las DevOps: el poder de los equipos multifuncionales - Adaptavist, fecha de acceso: febrero 27, 2026,
<https://www.adaptavist.com/es-es/blog/mejorar-agilidad-devops-equipos-multifuncionales>
36. ¿Qué es un ingeniero de DevOps? - Atlassian, fecha de acceso: febrero 27, 2026,
<https://www.atlassian.com/es/devops/what-is-devops/devops-engineer>
37. ¿Qué es DevOps? Funciones, responsabilidades y funcionamiento de los equipos DevOps. | Right People Group, fecha de acceso: febrero 27, 2026,
<https://rightpeoplegroup.com/es/blog/funciones-y-responsabilidades-clave-de-un-equipo-devops>
38. SRE SLOs: Defining SLAs, SLIs, and SLOs in SRE | NetApp, fecha de acceso: febrero 27, 2026,
<https://www.netapp.com/learn/cvo-blg-sre-slos-defining-slas-slis-and-slos-in-sre/>
39. Docs as Code — Write the Docs, fecha de acceso: febrero 27, 2026,
<https://www.writethedocs.org/guide/docs-as-code/>
40. Docs as Code — Write the Docs, fecha de acceso: febrero 27, 2026,
<https://www.writethedocs.org/guide/docs-as-code/#versioning-and-branching>
41. Roles admitidos para el desarrollo de software - Azure DevOps | Microsoft Learn, fecha de acceso: febrero 27, 2026,
<https://learn.microsoft.com/es-es/azure/devops/user-guide/roles?view=azure-devops>

42. Devops Engineer Assistant Vice President | Citi Careers, fecha de acceso: febrero 27, 2026,
<https://jobs.citi.com/job/pune/devops-engineer-assistant-vice-president/287/92193542992>
43. ¿Qué es un pipeline de CI/CD? - GitLab, fecha de acceso: febrero 27, 2026,
<https://about.gitlab.com/es/topics/ci-cd/cicd-pipeline/>
44. Integración continua - Documentación de GitHub, fecha de acceso: febrero 27, 2026, <https://docs.github.com/es/actions/get-started/continuous-integration>
45. Transforma tu flujo de trabajo con CI/CD: Cómo GitLab nos ayudó a automatizar despliegues y mejorar la colaboración en equipo, fecha de acceso: febrero 27, 2026,
<https://www.andresmartinezsoto.eu/automatizacion-despliegues-ci-cd-gitlab-colaboracion-equipo/>
46. Tutorial: Implementación de entornos en CI/CD mediante GitHub - Azure Deployment Environments | Microsoft Learn, fecha de acceso: febrero 27, 2026,
<https://learn.microsoft.com/es-es/azure/deployment-environments/tutorial-deploy-environments-in-cicd-github>
47. Implementación y actualización sin tiempo de inactividad: ¿qué debe saber? - WeblinIndia, fecha de acceso: febrero 27, 2026,
<https://www.weblinindia.com/es/blog/zero-downtime-deployment-upgrade/>
48. SLOs, SLIs, and SLAs: Meanings & Differences - New Relic, fecha de acceso: febrero 27, 2026, <https://newrelic.com/blog/observability/what-are-slos-slis-slals>
49. What are SLOs, SLAs, and SLIs? A complete guide to service reliability metrics - Incident.io, fecha de acceso: febrero 27, 2026, <https://incident.io/blog/slo-sla-sli>
50. How can I get into a DevSecOps career? - cybersecurity - Reddit, fecha de acceso: febrero 27, 2026,
https://www.reddit.com/r/cybersecurity/comments/1hjrf3p/how_can_i_get_into_a_devsecops_career/
51. DevOps observability: A guide for DevOps and DevSecOps teams - Dynatrace, fecha de acceso: febrero 27, 2026,
<https://www.dynatrace.com/news/blog/devops-observability-guide-for-devops-and-devsecops/>
52. Azure DevOps Document Management - Modern Requirements, fecha de acceso: febrero 27, 2026,
<https://www.modernrequirements.com/es/blogs/azure-devops-document-management-system/>
53. Making Documentation Simpler and Practical: Our Docs-as-Code Journey - Squarespace Engineering Blog, fecha de acceso: febrero 27, 2026,
<https://engineering.squarespace.com/blog/2025/making-documentation-simpler-and-practical-our-docs-as-code-journey>
54. ¿Cómo procedo correctamente al versionado de documentos? - PaperOffice, fecha de acceso: febrero 27, 2026,
<https://start.paperoffice.com/es/versiones-archivos-legalmente-beneficios-document-management-documentos-dms>
55. Eliminar el tiempo de inactividad a través de actualizaciones de servicios

versionadas - Azure DevOps | Microsoft Learn, fecha de acceso: febrero 27, 2026,
<https://learn.microsoft.com/es-es/devops/operate/achieving-no-downtime-versioned-service-updates>